Module 7

Real-World Applications

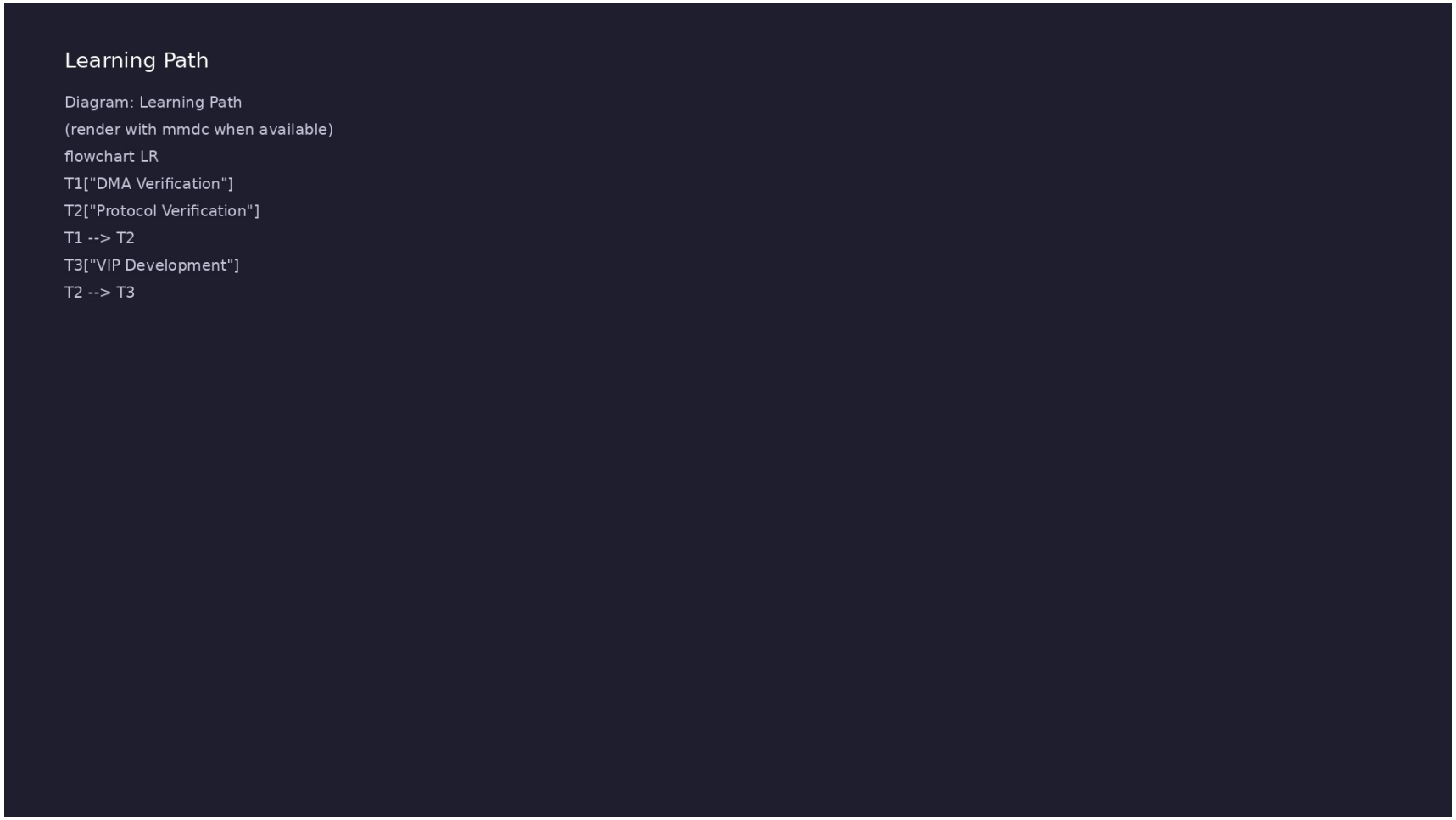
Learning objectives

- Apply UVM to real-world verification scenarios

Prerequisites

- See module README

Learning path



Apply UVM to real-world verification scenarios

Overview

- This module applies UVM to real-world verification scenarios including DMA verification, protocol v…

Syllabus topics

1. DMA Verification (1/9)

- DMA Controller Verification
- DMA transaction modeling
- DMA protocol implementation
- DMA sequence generation
- DMA result checking
- DMA Patterns

1. DMA Verification (2/9)

- DMA transfer patterns
- Multi-channel DMA
- DMA completion handling
- DMA error handling
- Interface (`dma_if`): DMA interface with start, done, addresses, length, and channel signals
- Transaction (`DmaTxn`): DMA transaction with source address, destination address, length, and chann…

1. DMA Verification (3/9)

- Sequence (`DmaSeq`): DMA sequence generating random DMA transfers
- Driver (`DmaDriver`): DMA driver implementing DMA protocol
- Environment (`DmaEnv`): DMA verification environment
- Test (`DmaTest`): DMA verification test with simple DMA DUT
- Method Signature: `function bit randomize();`
- Purpose: Generate random DMA transfer parameters

1. DMA Verification (4/9)

- Constraints: Use `with { constraint; }` to limit values
- Key: `len inside {[1:20]}` constrains transfer length
- Method Signature:
- `task get_next_item(ref T item);` - Get transaction from sequencer
- `task item_done();` - Signal transaction completion
- Purpose: Implement DMA protocol with start/done handshake

1. DMA Verification (5/9)

- Protocol Flow:
- Key: `do-while` loop waits for `dma_done` pulse (blocking wait)
- Timing: Start signal is one clock cycle pulse
- Blocking Behavior: `get_next_item()` blocks until sequencer provides transaction, `item_done()` unb...
- Completion Detection: Wait for `dma_done` signal assertion (may take multiple clock cycles)
- Method Signature: `task body();` (no parameters, no return)

1. DMA Verification (6/9)

- Purpose: Generate and send multiple DMA transfers to sequencer
- Execution Flow:
- Key: `repeat` loop creates multiple transactions sequentially
- Blocking Behavior:
- `start_item()` blocks until sequencer grants access
- `finish_item()` blocks until driver completes transaction (may take many cycles)

1. DMA Verification (7/9)

- Transaction Lifetime:
- Created in loop
- Sent to sequencer via `start_item()`
- Driver processes it (drives signals, waits for DMA done)
- Driver signals completion via `item_done()`
- `finish_item()` returns, loop continues

1. DMA Verification (8/9)

- Constraint: `len inside {[1:20]}` ensures reasonable transfer lengths
- DMA transaction modeling captures transfer semantics
- DMA protocol implementation handles start/done handshake
- DMA sequence generation creates realistic transfer scenarios
- DMA result checking verifies transfer completion
- DMA Protocol: Start pulse → Wait for done → Completion

1. DMA Verification (9/9)

- Transaction Flow: Sequence → Sequencer → Driver → DUT
- Completion Detection: Wait for `dma_done` signal assertion
- DMA transaction generation
- DMA protocol execution
- DMA transfer completion
- UVM report summary

2. Protocol Verification (1/9)

- UART Verification
- UART protocol modeling
- Serial communication verification
- Loopback testing
- UART-specific patterns
- SPI Verification

2. Protocol Verification (2/9)

- SPI protocol verification patterns
- UVM scaffold for SPI
- Protocol-specific testbench structure
- I2C Verification
- I2C protocol verification patterns
- UVM scaffold for I2C

2. Protocol Verification (3/9)

- Protocol-specific testbench structure
- Interface (`uart_if`): UART interface with TX/RX signals and control
- Transaction (`UartTxn`): UART transaction with data field
- Sequence (`UartSeq`): UART sequence generating random UART transfers
- Driver (`UartDriver`): UART driver implementing UART protocol
- Environment (`UartEnv`): UART verification environment

2. Protocol Verification (4/9)

- Test (`UartTest`): UART verification test with UART DUT
- Method Signature:
- `task get_next_item(ref T item);` - Get transaction from sequencer
- `task item_done();` - Signal transaction completion
- Purpose: Implement UART TX protocol
- Protocol Flow:

2. Protocol Verification (5/9)

- Key: Wait for `tx_busy` to deassert (transmission complete)
- Timing: Start signal is one clock cycle pulse
- Blocking Behavior: `get_next_item()` blocks until sequencer provides transaction, `item_done()` unb...
- Completion Detection: Wait for `tx_busy` signal deassertion (may take many clock cycles for serial...
- UART Characteristics: Serial transmission takes time (one bit per baud period)
- Purpose: Enable loopback testing

2. Protocol Verification (6/9)

- Pattern: `assign rx = tx` creates loopback path
- Use Case: Minimal smoke testing without external device
- UART protocol modeling captures serial communication
- Serial communication verification tests TX/RX paths
- Loopback testing enables minimal smoke testing
- Protocol-specific patterns provide reusable verification components

2. Protocol Verification (7/9)

- UART Protocol: Data → Start → Wait for busy → Completion
- Loopback Pattern: TX output connected to RX input
- Busy Signal: Wait for `tx_busy` deassertion
- UART transaction generation
- UART protocol execution
- UART TX/RX operation

2. Protocol Verification (8/9)

- Loopback verification
- Test (`SpiExampleTest`): SPI verification test scaffold
- UVM Pattern: Demonstrates UVM structure for SPI verification
- SPI protocol verification patterns provide reusable structure
- UVM scaffold for protocol verification enables rapid development
- Protocol-specific testbench structure supports future DUT integration

2. Protocol Verification (9/9)

- Test (`I2cExampleTest`): I2C verification test scaffold
- UVM Pattern: Demonstrates UVM structure for I2C verification
- I2C protocol verification patterns provide reusable structure
- UVM scaffold for protocol verification enables rapid development
- Protocol-specific testbench structure supports future DUT integration

3. VIP Development (1/6)

- VIP Architecture
- VIP component organization
- VIP packaging patterns
- VIP reuse strategies
- VIP configuration
- VIP Components

3. VIP Development (2/6)

- Transaction classes
- Sequence classes
- Driver and monitor
- Agent packaging
- Transaction (`VipTxn`): VIP transaction with payload
- Sequence (`VipSeq`): VIP sequence generating
  transactions

3. VIP Development (3/6)

- Driver (`VipDriver`): VIP driver
- Agent (`VipAgent`): VIP agent containing sequencer and driver
- Test (`VipTest`): VIP test demonstrating VIP usage
- Purpose: Package VIP components (sequencer + driver) in agent
- Pattern: Agent encapsulates all VIP components
- Key: Standard UVM agent structure for reusability

3. VIP Development (4/6)

- Purpose: VIP driver with structured logging
- Logging: Use `uvm_info` with formatted messages
- Key: `$sformatf()` for formatted string generation
- Purpose: Demonstrate VIP usage in test
- Pattern: Create VIP agent, use its sequencer
- Key: VIP agent is reusable component

3. VIP Development (5/6)

- VIP architecture enables reusable verification components
- VIP component organization provides clear structure
- VIP packaging patterns support easy integration
- VIP reuse strategies improve productivity
- VIP Packaging: Transaction + Sequence + Driver + Agent
- Reusability: VIP agent can be instantiated in any test

3. VIP Development (6/6)

- Structured Logging: Use `uvm_info` with formatted messages
- VIP transaction generation
- VIP agent operation
- VIP packaging demonstration
- VIP reuse pattern

4. Best Practices (1/10)

- Code Organization
- Component organization
- File structure
- Module organization
- Package organization
- Naming Conventions

4. Best Practices (2/10)

- Component naming
- Variable naming
- File naming
- Test naming
- Logging and Debugging
- Structured logging

4. Best Practices (3/10)

- Phase-based logging
- Topology visualization
- Debug techniques
- Test (`BestPracticesTest`): Test demonstrating best practices
- Phase Organization: Proper phase usage and logging
- Topology Printing: Component hierarchy visualization

4. Best Practices (4/10)

- Logging Patterns: Structured logging for maintainability
- Purpose: Log execution flow through phases
- Pattern: Log at start of each phase
- Key: Consistent logging improves debugging
- Method Signature:
- `static function uvm_root get();` → returns singleton root instance

4. Best Practices (5/10)

- `function void print_topology();` → prints hierarchy
- Purpose: Visualize component hierarchy for debugging
- Timing: Call in `end_of_elaboration_phase()` (after all connections)
- Output: Prints complete component tree with hierarchical paths
- Output Format:
- Key: Helps debug component structure and connections

4. Best Practices (6/10)

- When to Use:
- After all components created
- After all connections made
- When debugging component hierarchy issues
- To verify testbench structure
- Benefits:

4. Best Practices (7/10)

- Visual representation of testbench structure
- Helps identify missing components
- Verifies hierarchical naming
- Aids in debugging connection issues
- Method Signature: `function void end_of_elaboration_phase(uvm_phase phase);`
- Phase Type: Build-time phase (function, executes bottom-up)

4. Best Practices (8/10)

- Purpose: Final elaboration tasks after all connections
- Common Uses: Print topology, validate configuration, final checks
- Timing: After `connect_phase()`, before `start_of_simulation_phase()`
- Code organization improves maintainability
- Naming conventions enhance readability
- Phase-based logging provides clear execution flow

4. Best Practices (9/10)

- Topology visualization aids debugging
- Phase Logging: Log at start of each phase
- Topology Printing: Use `print_topology()` in `end_of_elaboration_phase()`
- Structured Logging: Use consistent message IDs and formats
- end_of_elaboration_phase: Use for final checks and topology printing
- Phase-based logging

4. Best Practices (10/10)

- Component topology printing
- Best practices demonstration
- Structured logging output

5. System-Level Verification (1/11)

- Multi-DUT Verification
- Multiple DUT instantiation
- Multi-DUT coordination
- System integration
- System-level testing
- Real-World Patterns

5. System-Level Verification (2/11)

- Practical verification scenarios
- Production-quality testbenches
- Maintainable verification environments
- Scalable verification solutions
- Method Signature: `function bit randomize();`
- Return Type: `bit` (1 = success, 0 = failure)

5. System-Level Verification (3/11)

- Purpose: Generate random DMA transfer parameters
- Usage: `void'(t.randomize() with { len inside {[1:20]}; });`
- Parameters: None (randomizes all `rand` fields in object)
- Constraints: Use `with { constraint_expression; }` to limit values
- Example: `len inside {[1:20]}` constrains transfer length to 1-20 bytes
- Key: `void'()` suppresses Verilator warning about ignored return value

5. System-Level Verification (4/11)

- Constraint Syntax:
- `len inside {[1:20]}` - Range constraint
- `len inside {1, 2, 4, 8, 16}` - Set constraint
- `len > 0 && len < 100` - Boolean constraint
- Randomization Order:
- Pattern: Start pulse → Wait for done → Completion

5. System-Level Verification (5/11)

- Key: `do-while` loop waits for `dma_done` pulse
- Pattern: Data → Start → Wait for busy deassert → Completion
- Key: Wait for `tx_busy` to deassert
- Event Control: `@(posedge clk)` - wait for positive clock edge
- Non-blocking Assignment: `<=` - schedule assignment for next time step
- Do-While Loop: `do @(posedge clk); while (condition);` - wait for condition

5. System-Level Verification (6/11)

- Purpose: Enable loopback testing
- Use Case: Minimal smoke testing without external device
- Purpose: Package VIP components for reusability
- Pattern: Agent encapsulates sequencer + driver
- Key: Standard UVM agent structure
- Macro: `uvm_info(id, message, verbosity)`

5. System-Level Verification (7/11)

- Function: `$sformatf(format, args)` → string
- Purpose: Log transaction with formatted message
- Format Specifiers:
- `%02h` - 2-digit hexadecimal, zero-padded
- `%08h` - 8-digit hexadecimal, zero-padded
- `%0d` - Decimal, no padding

5. System-Level Verification (8/11)

- `%s` - String
- Verbosity Levels: UVM_NONE, UVM_LOW, UVM_MEDIUM, UVM_HIGH, UVM_FULL
- Message IDs: Use consistent IDs for filtering and debugging
- Purpose: Log execution flow through phases
- Pattern: Log at start of each phase
- Key: Consistent logging improves debugging

5. System-Level Verification (9/11)

- Method Signature:
- `static function uvm_root get();` → returns singleton root
- `function void print_topology();` → prints hierarchy
- Purpose: Visualize component hierarchy
- Timing: Call in `end_of_elaboration_phase()` (after all connections)
- Output: Prints complete component tree with hierarchical paths

5. System-Level Verification (10/11)

- Usage:
- Phase Type: Build-time phase (function, executes bottom-up)
- Purpose: Final elaboration tasks after all connections
- Timing: After `connect_phase()`, before `start_of_simulation_phase()`
- Common Uses:
- Print topology

5. System-Level Verification (11/11)

- Validate configuration
- Final checks
- Setup verification

Hands-on examples

Module 7 self-check

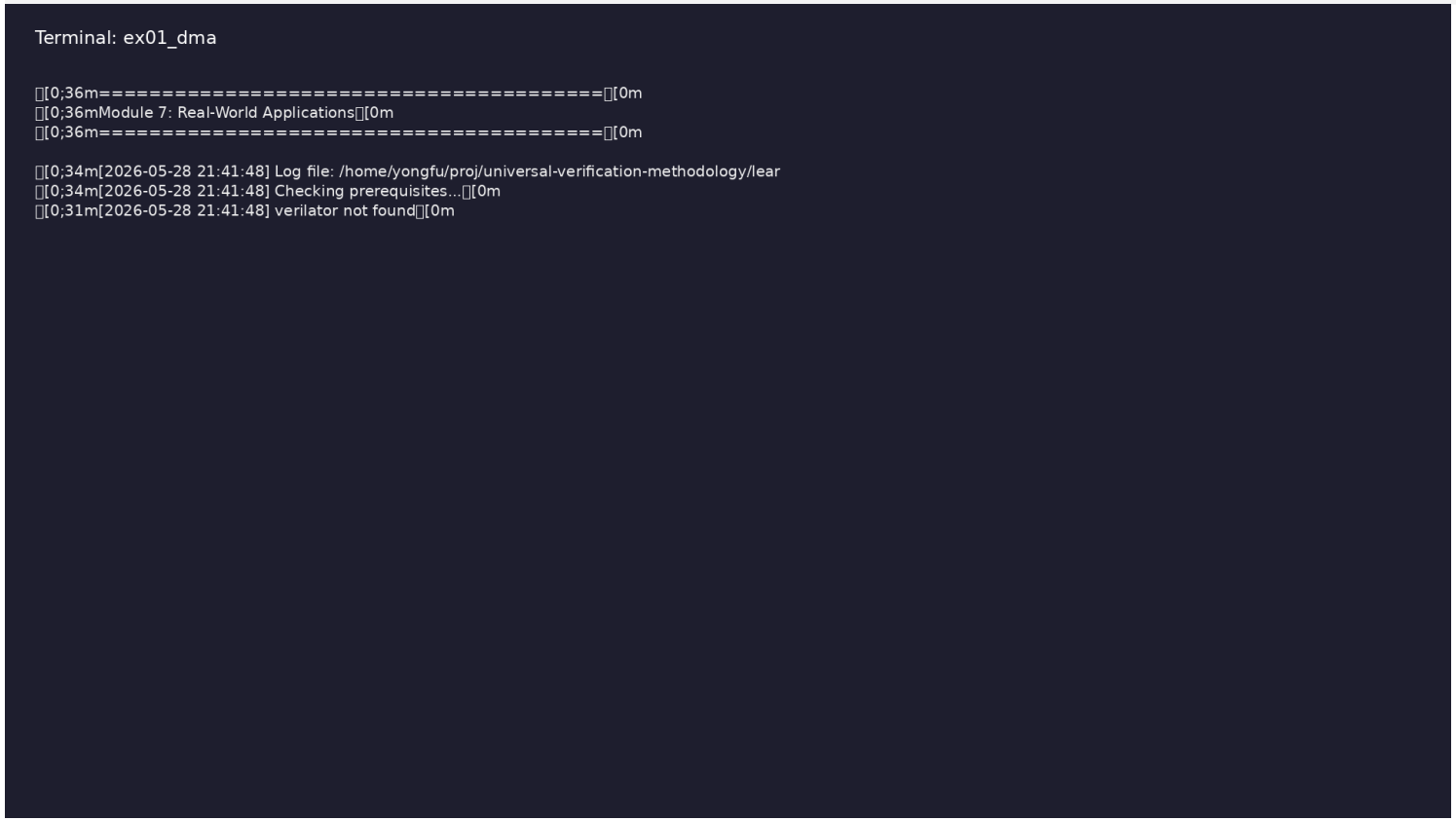
```
$ ./scripts/module7.sh --dma
```

Example 1: DMA Verification

- Interface (`dma_if`): DMA interface with start, done, addresses, length, and channel signals
- Transaction (`DmaTxn`): DMA transaction with source address, destination address, length, and chann…
- Sequence (`DmaSeq`): DMA sequence generating random DMA transfers

Demo: DMA Verification

```
$ ./scripts/module7.sh --dma
```

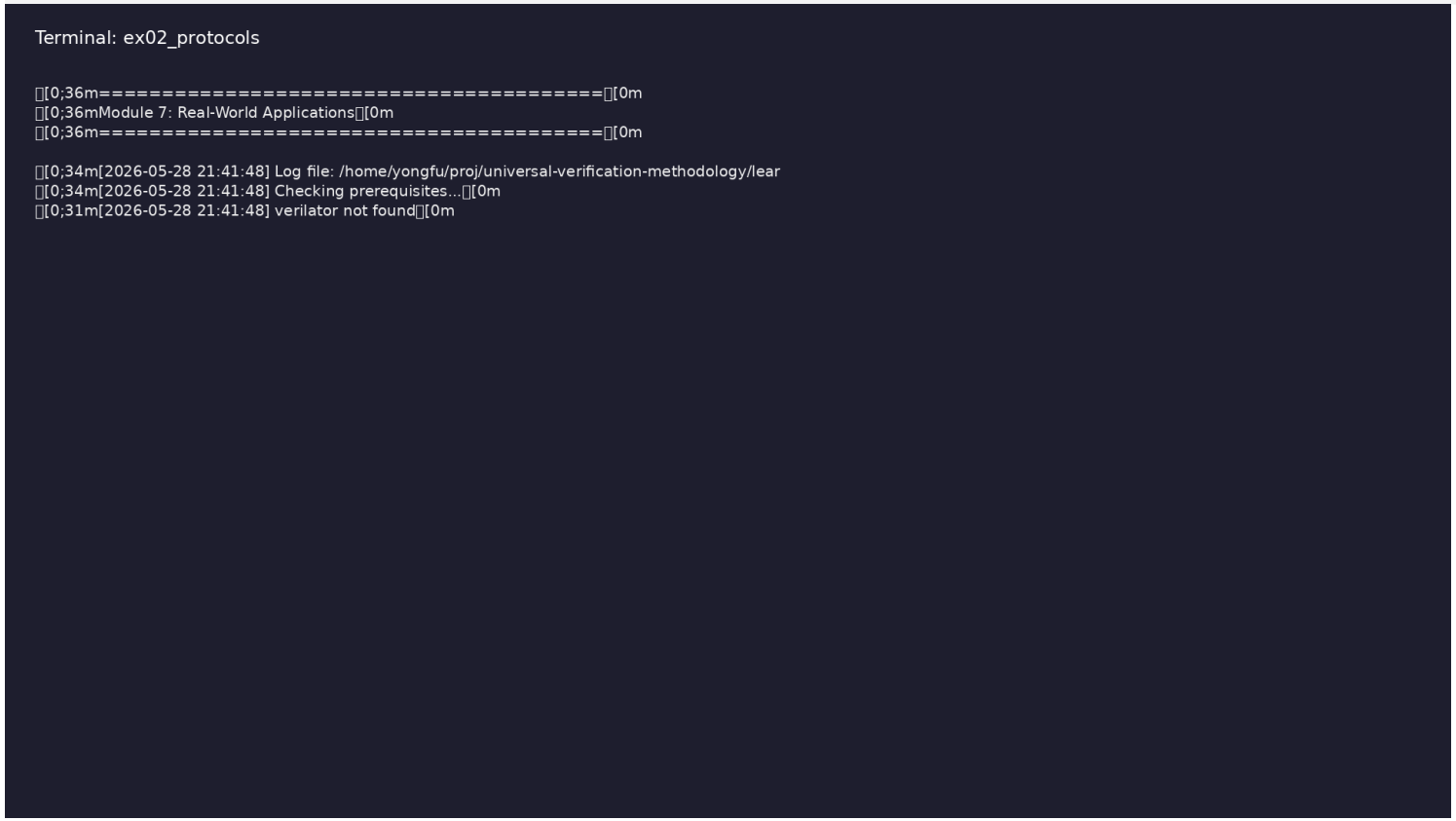


Example 2: UART Verification

- Interface (`uart_if`): UART interface with TX/RX signals and control
- Transaction (`UartTxn`): UART transaction with data field
- Sequence (`UartSeq`): UART sequence generating random UART transfers

Demo: UART Verification

```
$ ./scripts/module7.sh --protocols
```

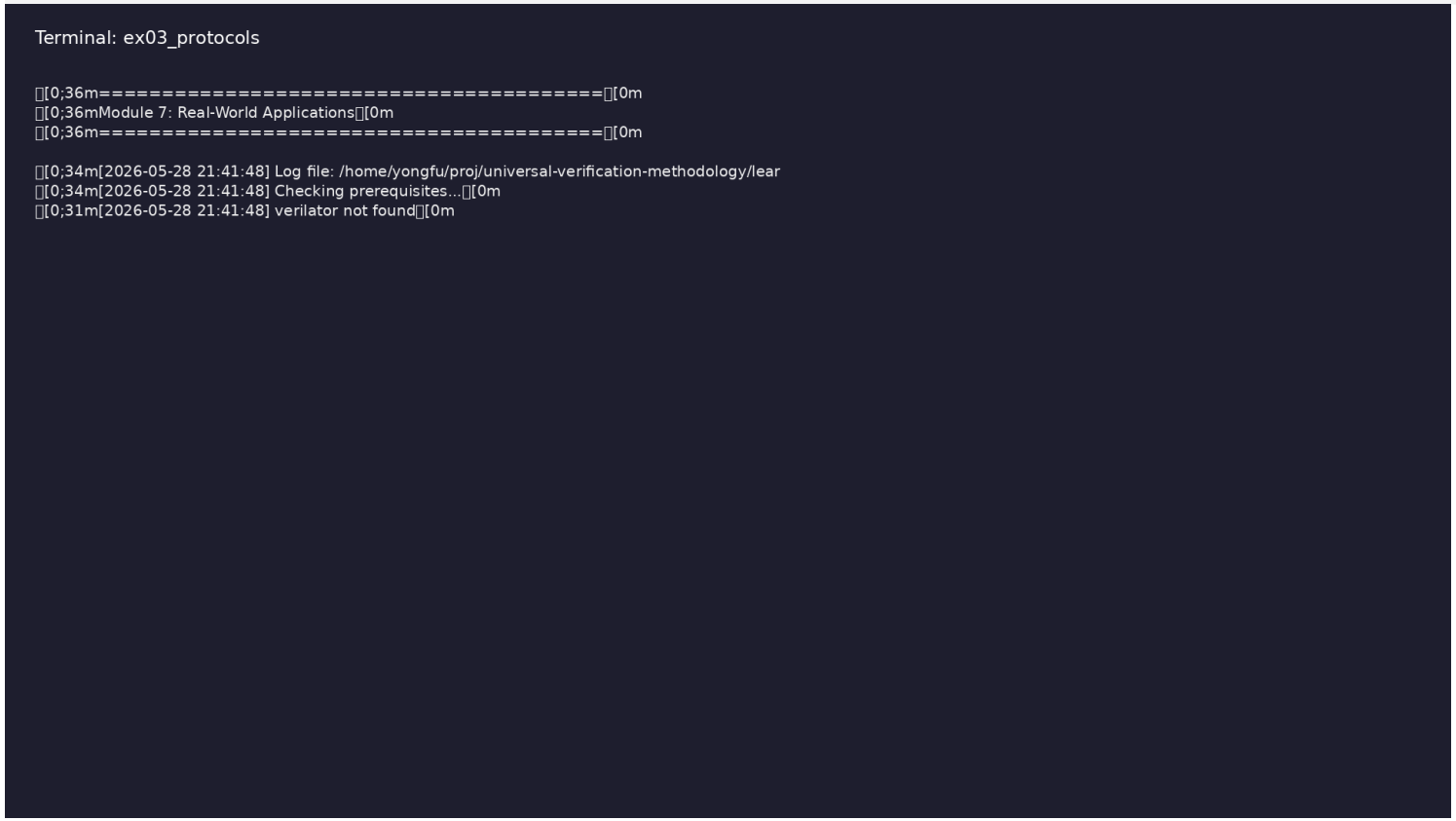


Example 3: SPI Verification

- Test (`SpiExampleTest`): SPI verification test scaffold
- UVM Pattern: Demonstrates UVM structure for SPI verification

Demo: SPI Verification

```
$ ./scripts/module7.sh --protocols
```

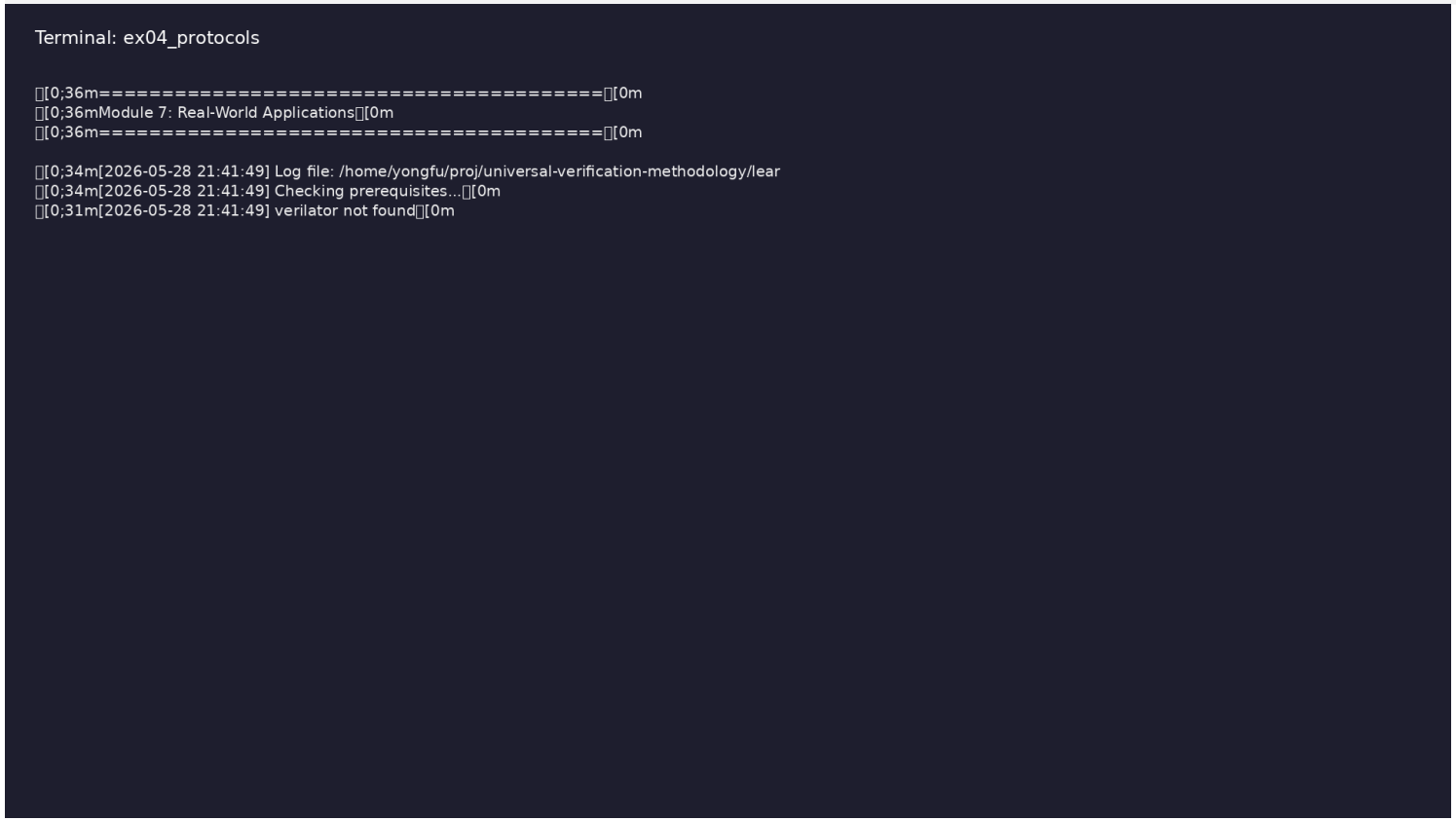


Example 4: I2C Verification

- Test (`I2cExampleTest`): I2C verification test scaffold
- UVM Pattern: Demonstrates UVM structure for I2C verification

Demo: I2C Verification

```
$ ./scripts/module7.sh --protocols
```

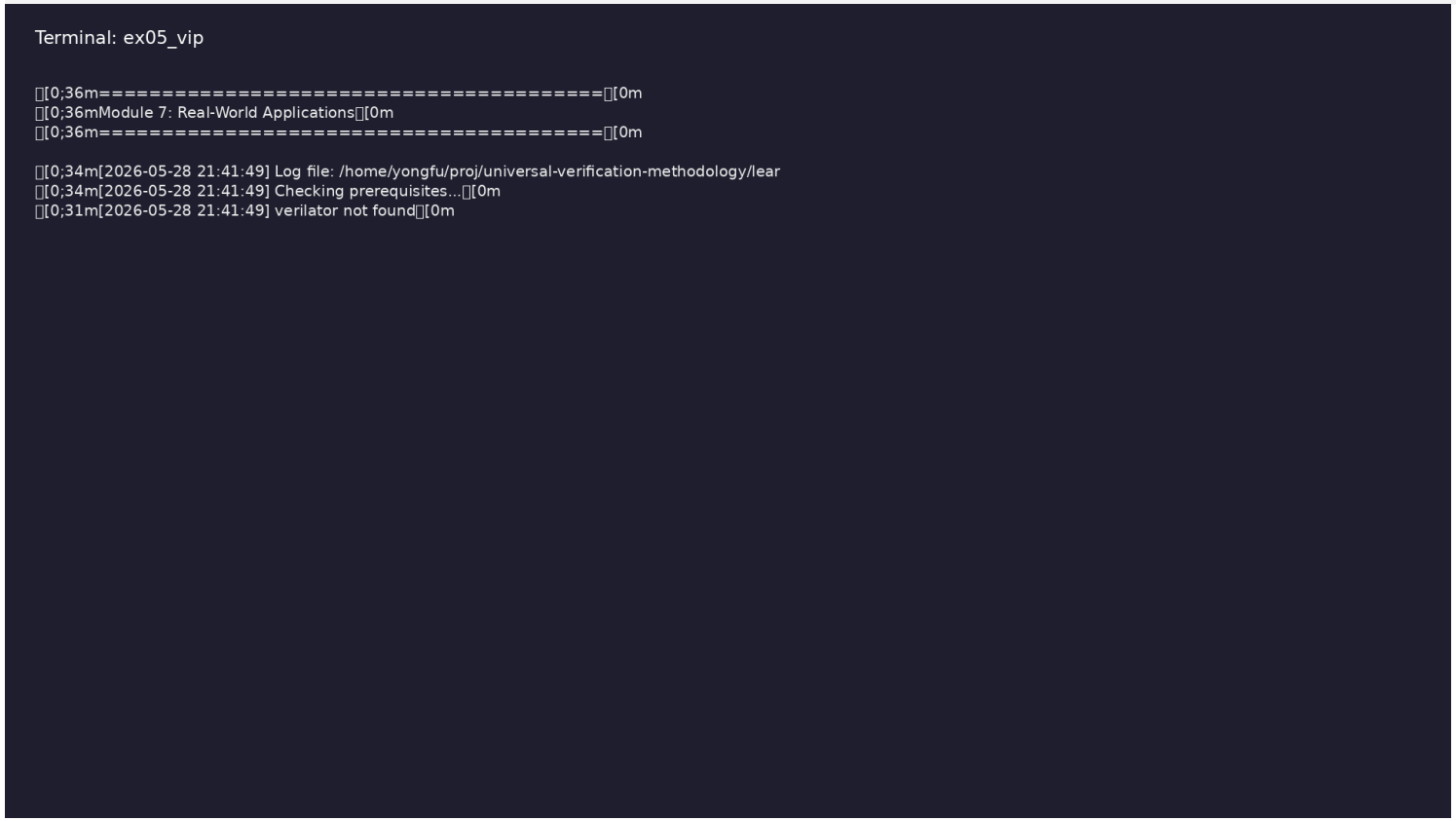


Example 5: VIP Development

- Transaction (`VipTxn`): VIP transaction with payload
- Sequence (`VipSeq`): VIP sequence generating transactions
- Driver (`VipDriver`): VIP driver

Demo: VIP Development

```
$ ./scripts/module7.sh --vip
```

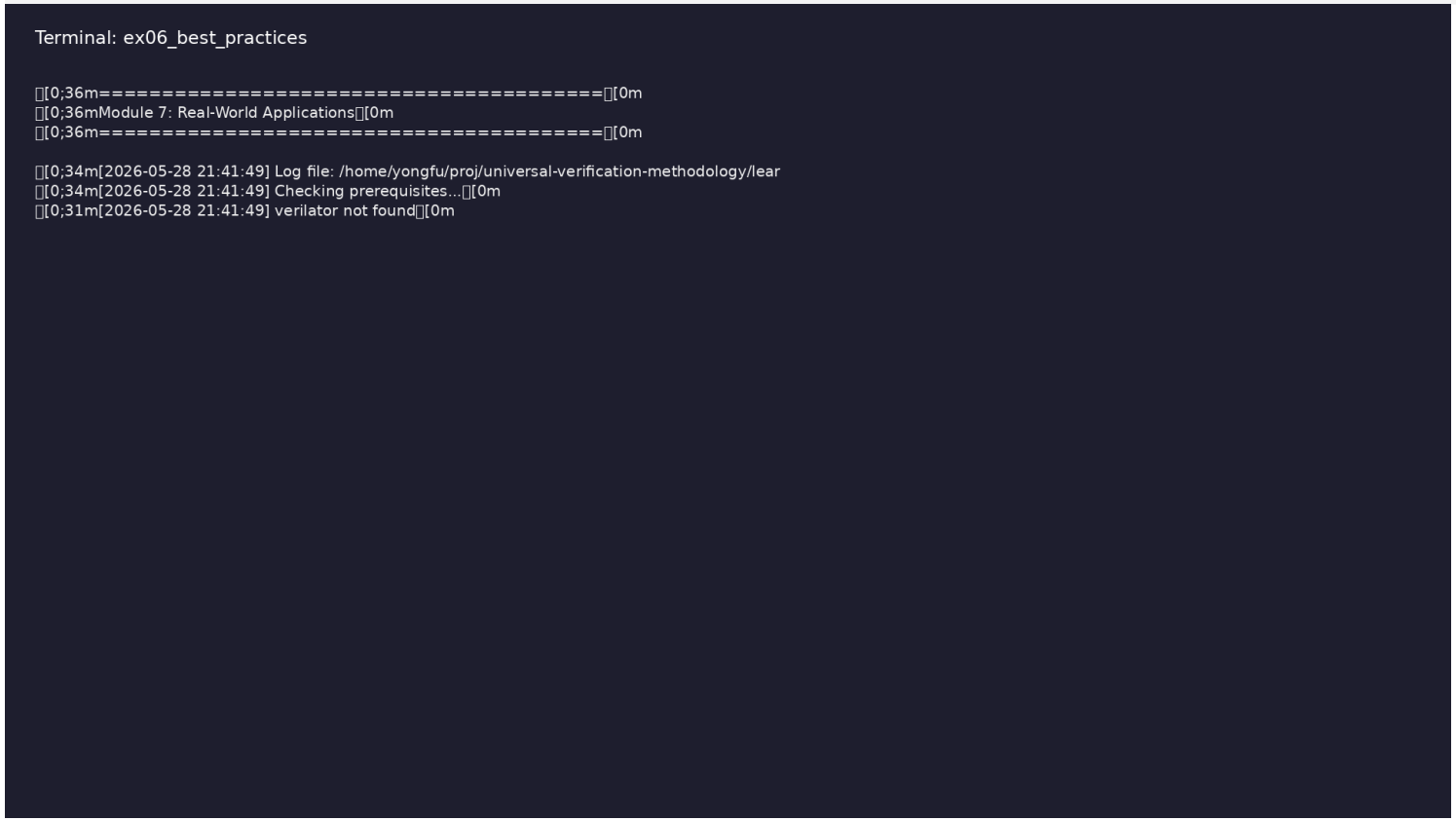


Example 6: Best Practices

- Test (`BestPracticesTest`): Test demonstrating best practices
- Phase Organization: Proper phase usage and logging
- Topology Printing: Component hierarchy visualization

Demo: Best Practices

```
$ ./scripts/module7.sh --best-practices
```



Practice & assessment

What you should know (1/2)

- Verify complex designs (DMA, protocols) using UVM patterns
- Develop reusable VIP with proper architecture
- Apply best practices for code organization and maintainability
- Perform system-level verification with multiple DUTs
- Create production-quality testbenches
- Understand protocol verification patterns

What you should know (2/2)

- Package verification components for reuse
- Use topology printing for debugging
- Implement phase-based logging
- Apply real-world verification patterns

Exercises

- DMA Verification
- Protocol Verification
- VIP Development
- Best Practices
- System-Level Verification

Assessment checklist

- Can verify complex designs (DMA, protocols)
- Can develop reusable VIP
- Can apply best practices
- Can perform system-level verification
- Can create production-quality testbenches
- Understands protocol verification patterns

Summary & next steps

- Apply UVM to real-world verification scenarios
- Complete module7/CHECKLIST.md
- Review module7/EXAMPLES.md and run each lab
- Next: Next module in course